

Algorithms and Data Structures for Data Science

Sorting Algorithms and Recursion

CS 277

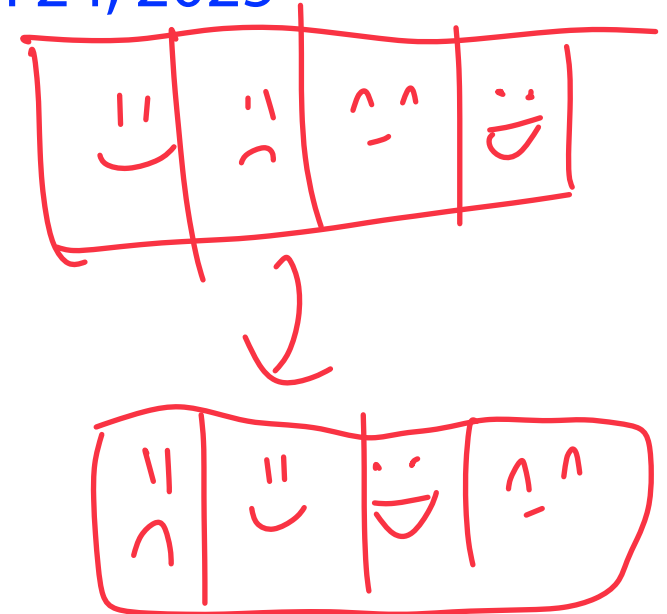
Brad Solomon

March 24, 2025



UNIVERSITY OF
ILLINOIS
URBANA - CHAMPAIGN

Department of Computer Science



Announcements

MP_automata is due this Wednesday!

MP_genomics releases on Wednesday! (3 week MP!)

Fill out IEF feedback form for +4 EC (collectively)



Learning Objectives

A brief overview of class progress

Motivate and discuss the need for sorted data structures

Introduce common sorting algorithms

Practice **Big O and code reading** in the context of algorithms

Introduce recursion

What will you get out of this class?

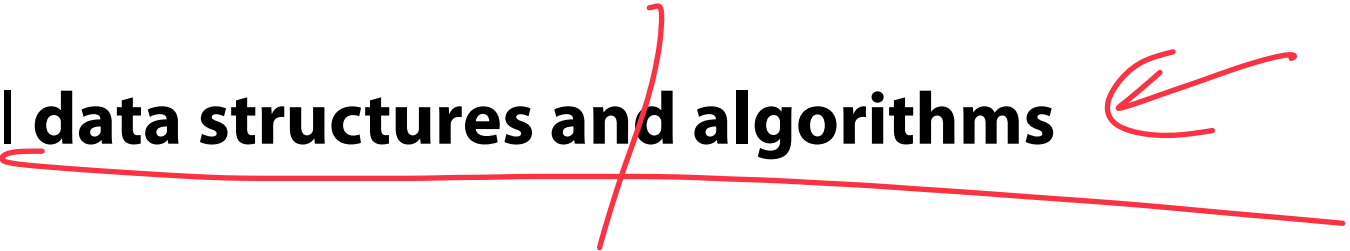
Build up a foundation of programming skills for future work

A red horizontal underline is drawn under the text. A red arrow starts from the right side of the underline and points downwards towards the next line of text.

Navigate, organize, and run moderately complex Python projects

A red horizontal underline is drawn under the text.

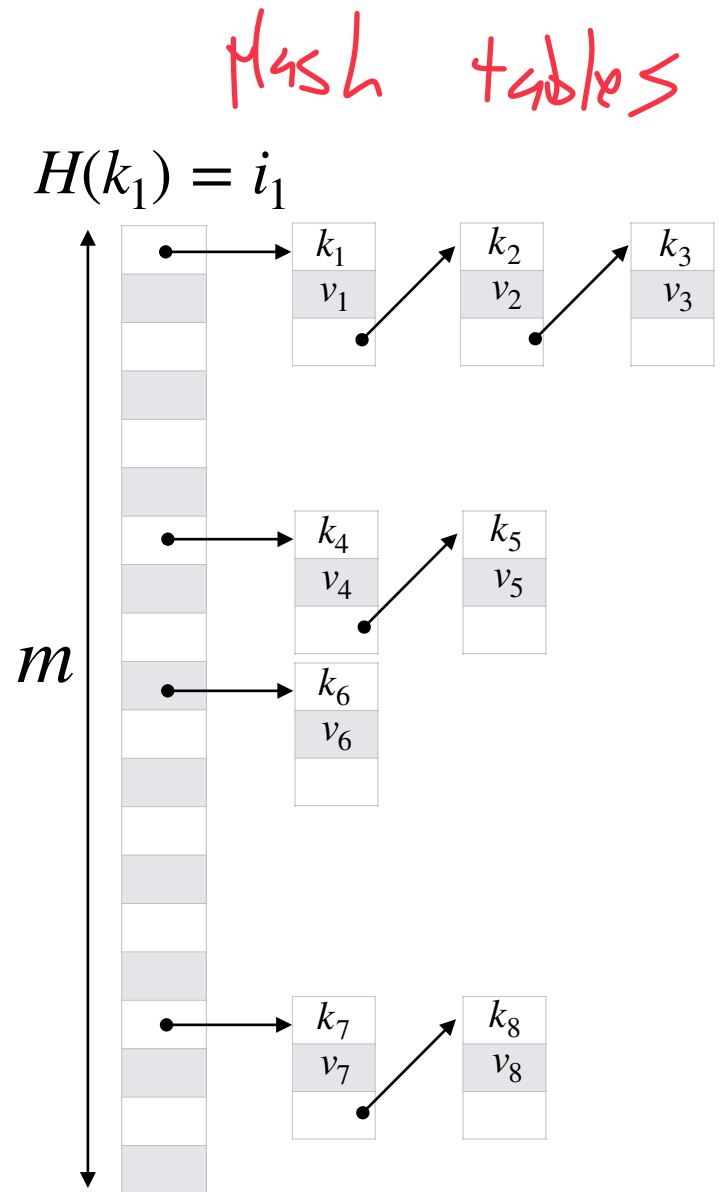
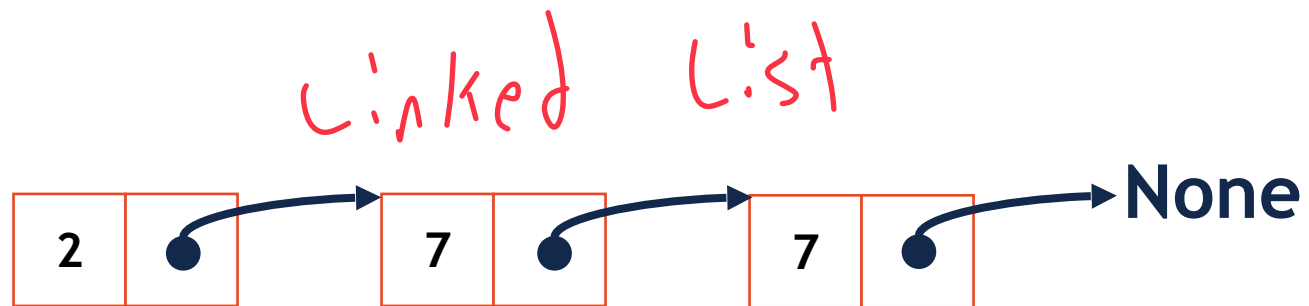
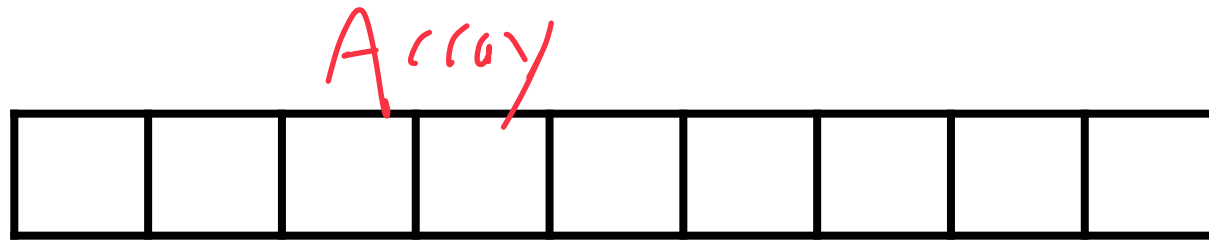
Understand foundational **data structures and algorithms**

A red horizontal underline is drawn under the text. A red arrow starts from the right side of the underline and points upwards towards the text.

Justify appropriate algorithms for data science problems

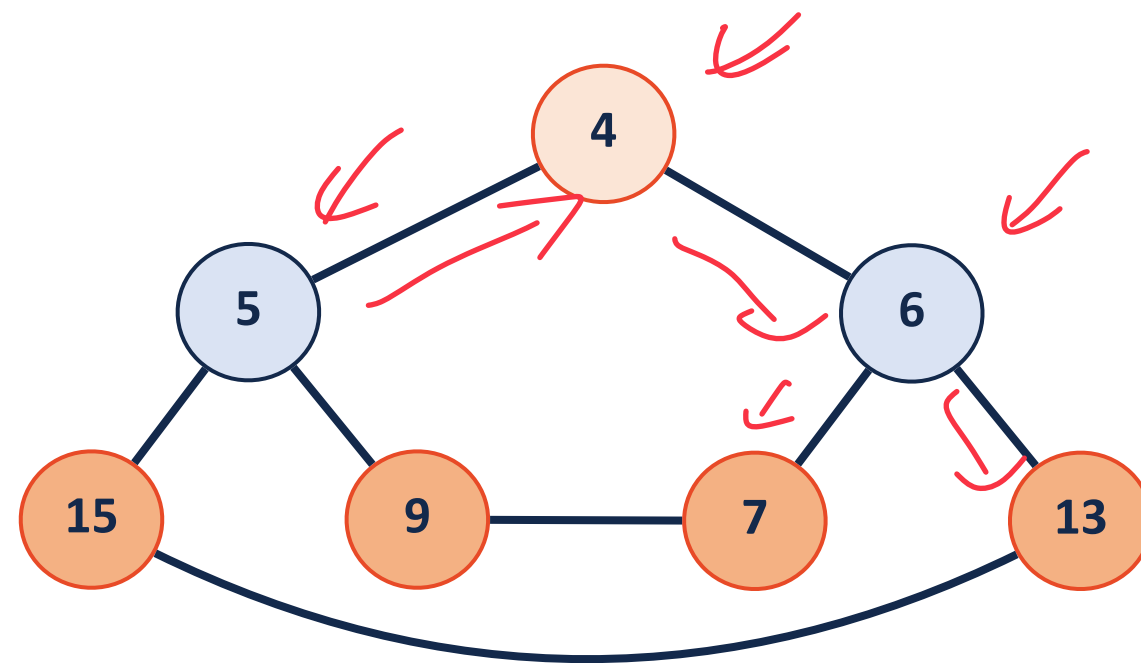
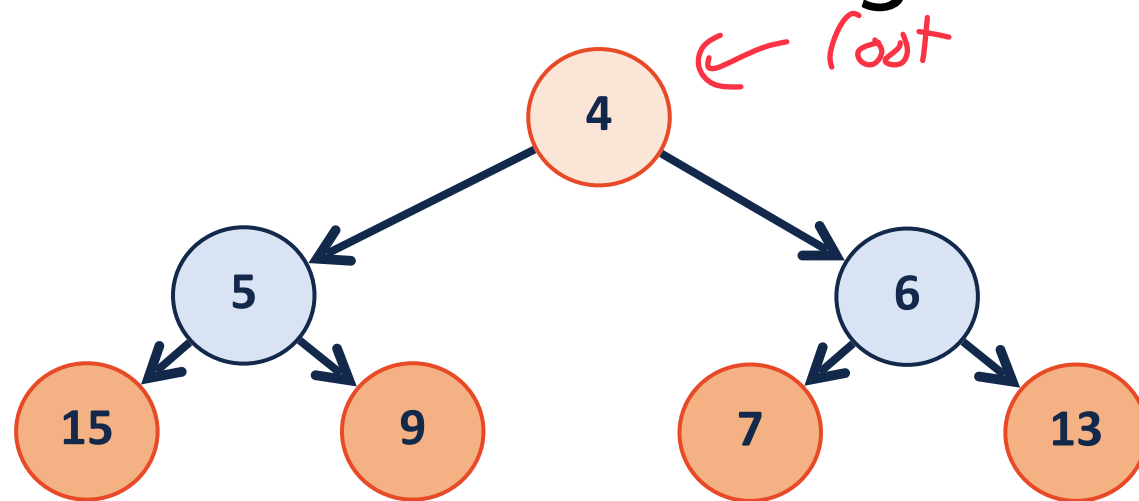
A red arrow starts from the right side of the text and points upwards towards the text. Below the arrow, the handwritten text "B'g O" is written in red.

Data Structures so far...



Data Structures to go...

Non-linear!



Algorithms are important too!

Trees and graphs both have more complex algorithms / implementations

To help us prepare, this week we will see some useful **sorting** algorithms

Good practice for:

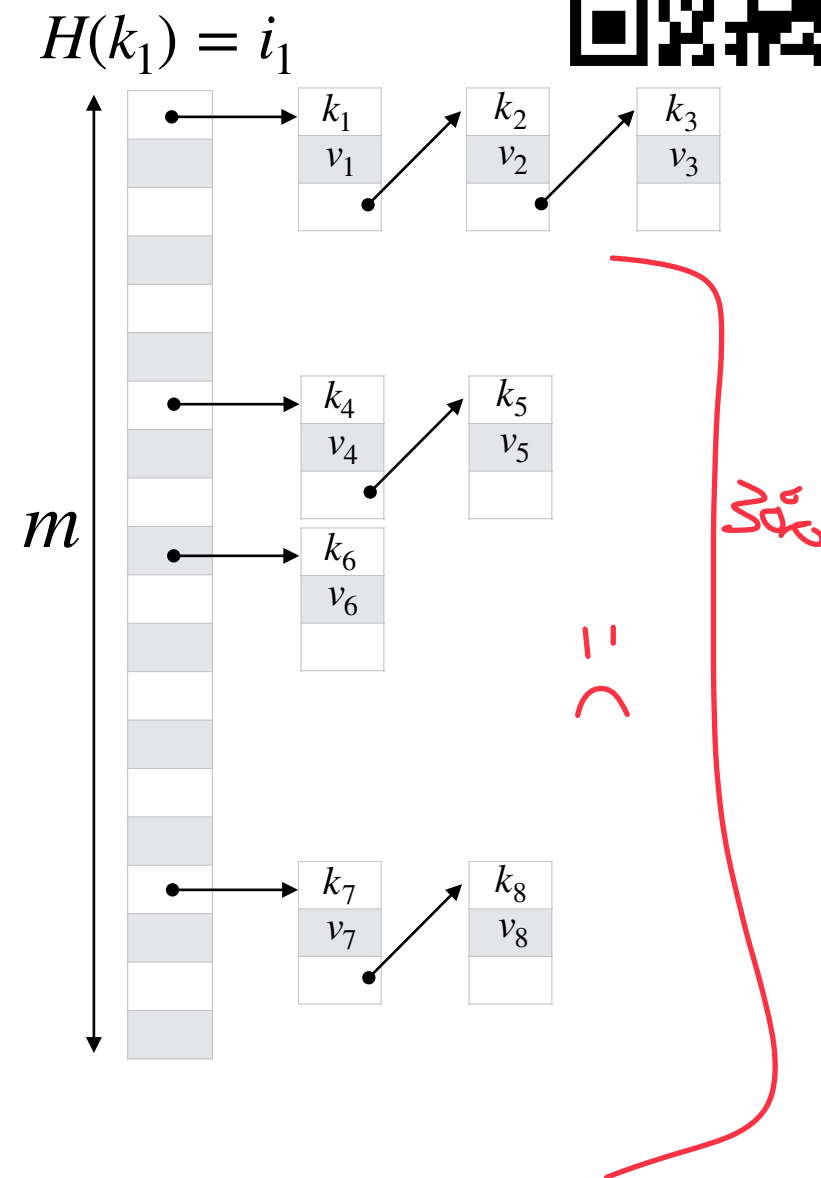
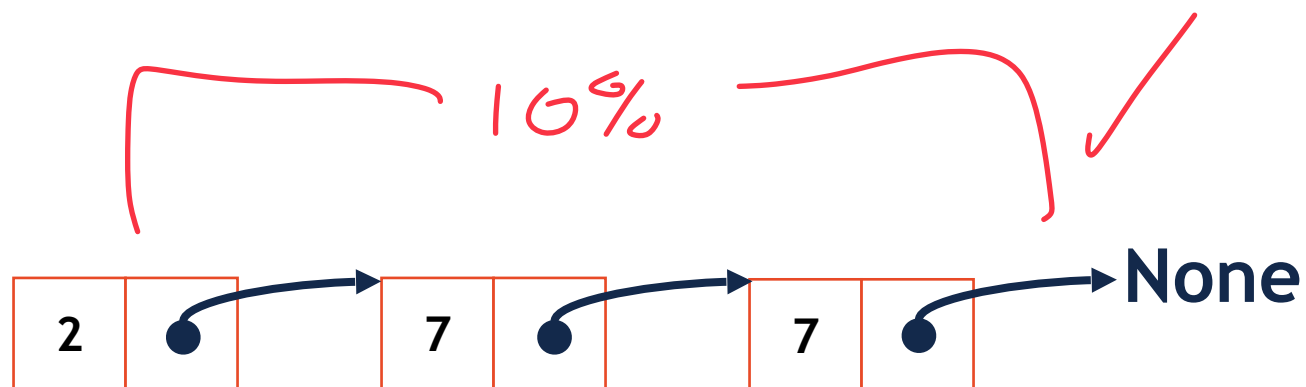
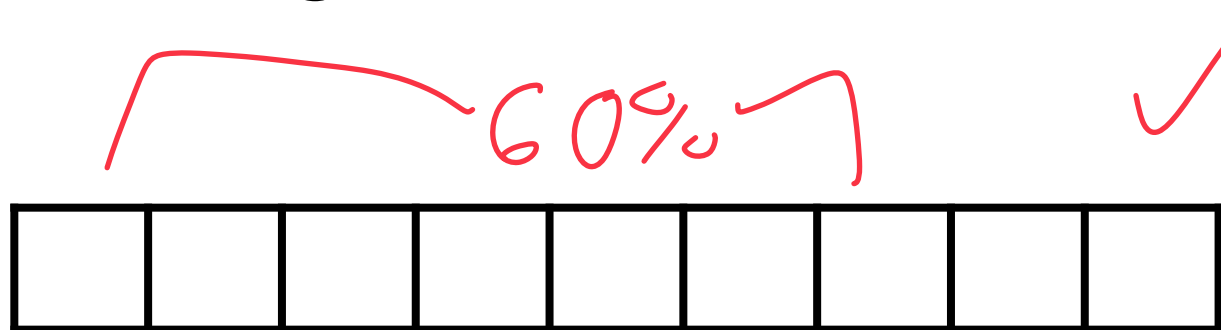
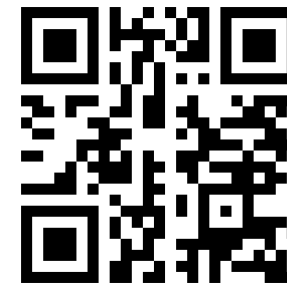
↳ Code reading

↳ Big O

↳ Computational problem solving

Sorting algorithms are also useful in their own right!

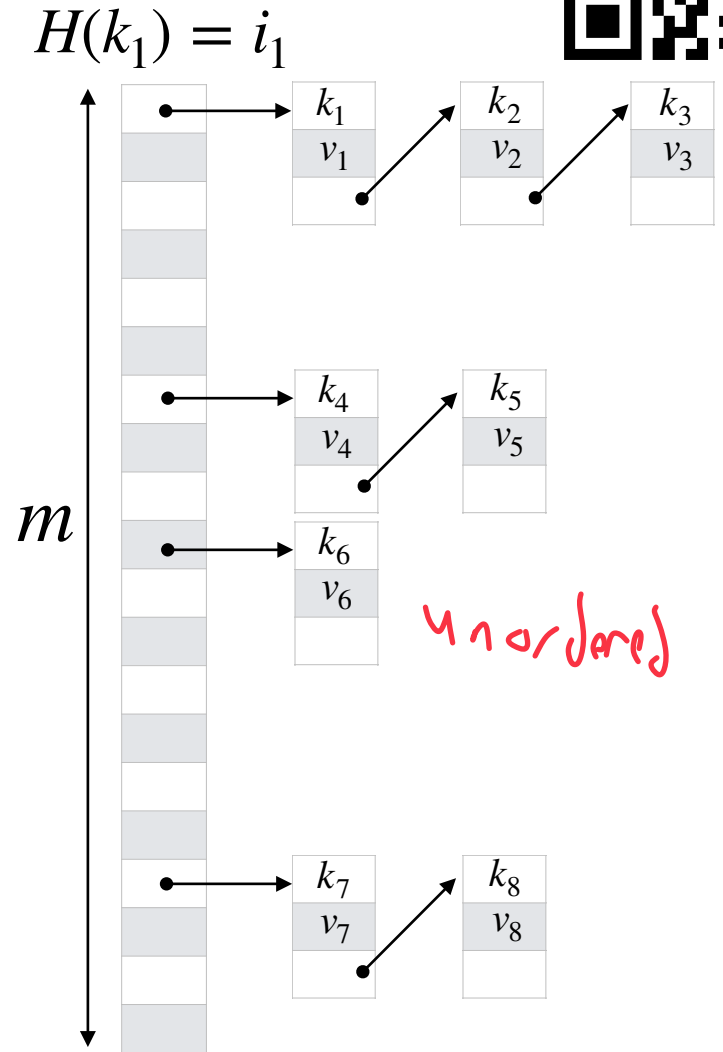
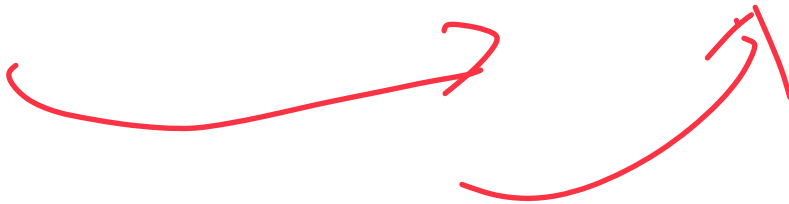
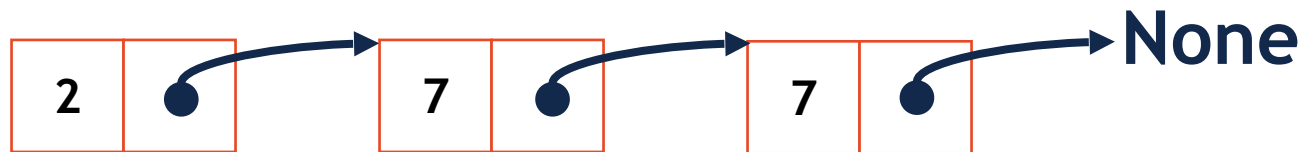
Is sorting useful for these data structures?



Which data structures are **ordered**?



↑
ordered!



The dataset sorting problem



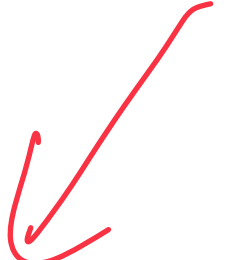
Given a collection of objects, C , with comparable values, order the objects such that $\forall x \in C, x_i \leq x_{i+1}$



this ' $\leq$ ' op exists

???

Array
Linked list



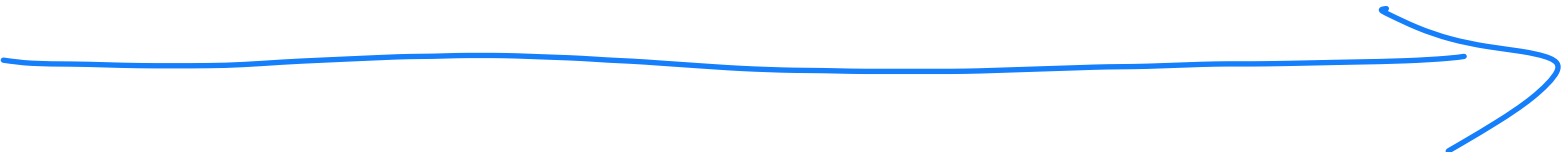
Input:

8	4	3	1	2	5	6	9	0	7
---	---	---	---	---	---	---	---	---	---

Output:

0	1	2	3	4	5	6	7	8	9
---	---	---	---	---	---	---	---	---	---

Sorting leads to efficient searching **Search (7)**



8	4	3	1	2	5	6	9	0	7
---	---	---	---	---	---	---	---	---	---

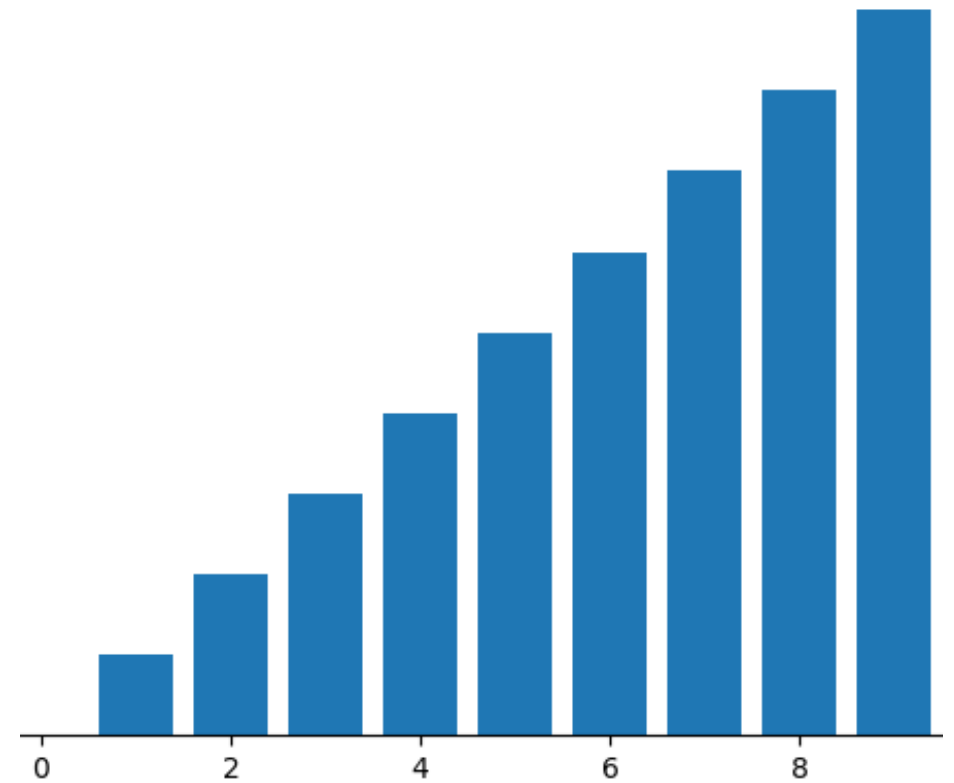
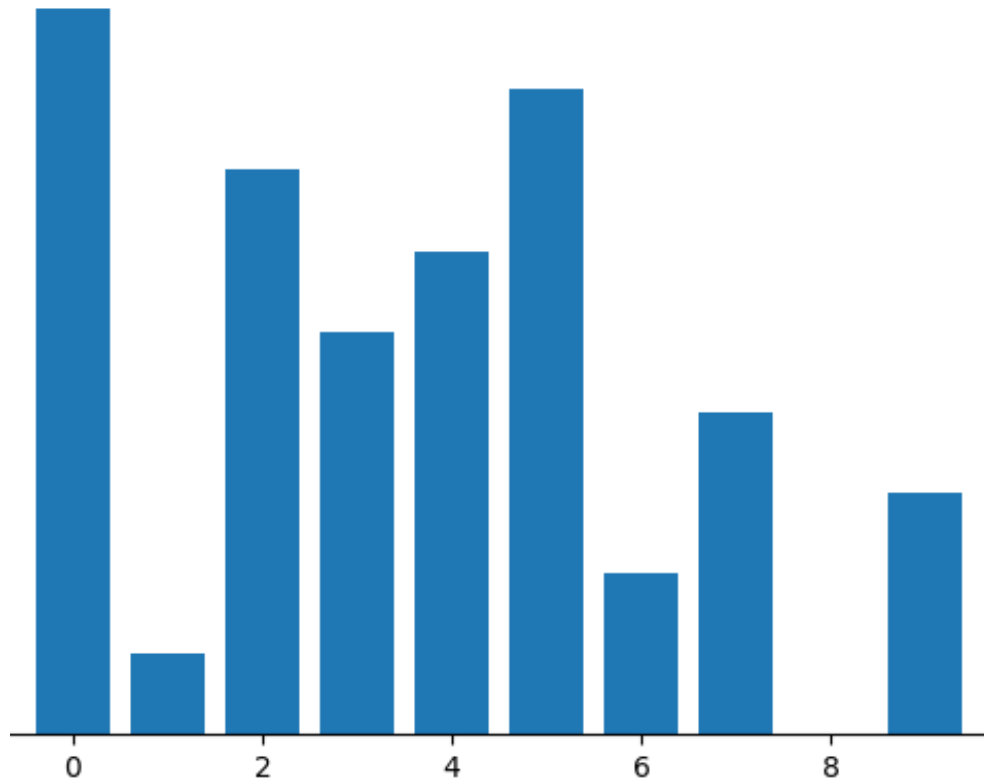
$O(n)$

0	1	2	3	4	5	6	7	8	9
---	---	---	---	---	---	---	---	---	---

Binary search

$O(\log n)$

Sorting leads to better visualization



Sorting is a fundamental problem in CS

Many algorithms begin with or include a sorting step

Fundamental sorting algorithms are great for mastering concepts

Sorting algorithms are a classic introduction to algorithms

Optimizing sort is an ongoing challenge

GraySort: Sort rate (TBs / minute) achieved while sorting a very large amount of data (currently 100 TB minimum).

CloudSort: Minimum cost (Dollars) for sorting a very large amount of data on a public cloud. (currently 100 TB).

MinuteSort: Amount of data that can be sorted in 60 seconds or less.

TeraByteSort: Elapsed time to sort 1 TB of data

Sorting in Python



We can sort any* list in Python by calling: `<list>.sort()`

```
1 intList = [4, 2, 1, 5]
2 floatList = [0.8, 0.1, 2.2, 3.3]
3 stringList = ["Apple", "Banana", "Balloons", "Aardvarks"]
4 listList = [intList, floatList, stringList]
5
6
7 for l in listList:
8     l.sort()
9
10 listList.sort()
11
12
13
```

↓ ↓
Aardvarks
Apple
Balloons
Banana
↑

Which of the following lists here **can't** be sorted? ... **Why?**

A) intList

C) stringList

← 20%

B) floatList

D) listList

← 80%

Sorting in Python



We can sort any list **of comparable objects** with `<list>.sort()`

```
1 l1 = [1, 2, 3]
2 l2 = [3, 2, 1]
3
4 print(l1 < l2)
5
```

And yes, lists can be compared with each other!

```
1 l1 = [1, 2, 3]
2 l2 = ["A", "B"]
3
4 print(l1 < l2)
5
```

What error message would this yield?

A) Value Error

C) Index Error

B) Type Error

D) Attribute Error

← 80%

Sorting Algorithms

Most of the time you can get away with the built-in sort...

... but you should understand the underlying implementation

As it turns out, Python's built-in sort is actually a hybrid sort called Timsort

It will pick between different sorting strategies based on the input!

Why is a heavily optimized sorting algorithm built this way? 

↳ there is no single best sort

Tradeoffs

Sorting Algorithms



As it turns out, there is no single 'best' sorting strategy! We will learn four:

SelectionSort

InsertionSort

MergeSort

QuickSort

As we introduce them, consider what is good / bad about each!

Sorting in place vs making a new list



Imagine you are given an unsorted list and want to sort it.

We can do this by changing the order of values in the existing list

This is known as **sorting in place**

We can do this by making a new list and copying values over in order

Which strategy would you prefer? **Why?**

40% Sort
in
place

↳ Efficient! Don't have to copy all values

60% Make a new list

↳ Visually seems easier!

SelectionSort

input list → sorted list out



```
1 def selectionSort(inList):
2     n = len(inList)
3
4     for i in range(n):
5         minindex = i
6         for j in range(i+1, n):
7             if inList[j] < inList[minindex]:
8                 minindex = j
9
10        inList[i], inList[minindex] = inList[minindex], inList[i]
```

Handwritten annotations on the code:

- Red arrow from "input list" to the `inList` parameter in line 1.
- Red arrow from "sorted list out" to the swap operation in line 10.
- Red arrow from "1st min value" to `minindex = i` in line 5.
- Red arrow from "2nd min value" to the inner loop in line 6.
- Red circle around `i` in line 10.

Which of the following best describes the selection sort algorithm?

- A) Repeatedly find the minimum values and adds them to a new list ~~X~~
- B) Starting at the first index, repeatedly swap until all items are in place
- C) Repeatedly find the min values and swap them into place

SelectionSort

```
1 def selectionSort(inList):
2     n = len(inList)
3
4     for i in range(n):
5         minindex = i
6         for j in range(i+1, n):
7             if inList[j] < inList[minindex]:
8                 minindex = j
9
10        inList[i], inList[minindex] = inList[minindex], inList[i]
```

1) Find min value

2) Swap min value w/ index i



8	5	1	3	4
---	---	---	---	---

2

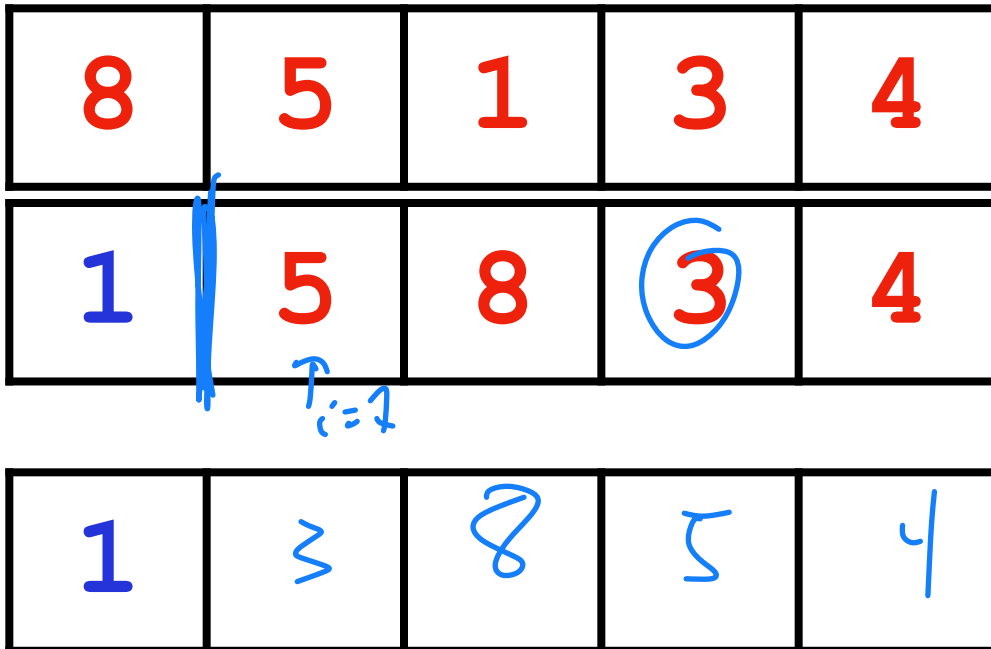
1	5	8	3	4
---	---	---	---	---

0

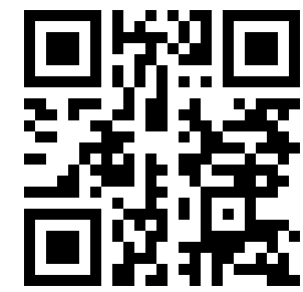
2

SelectionSort

```
1 def selectionSort(inList):
2     n = len(inList)
3
4     for i in range(n):
5         minindex = i
6         for j in range(i+1, n):
7             if inList[j] < inList[minindex]:
8                 minindex = j
9
10        inList[i], inList[minindex] = inList[minindex], inList[i]
```



SelectionSort



```
1 def selectionSort(inList):
2     n = len(inList)
3
4     for i in range(n):
5         minindex = i
6         for j in range(i+1, n):
7             if inList[j] < inList[minindex]:
8                 minindex = j
9
10        inList[i], inList[minindex] = inList[minindex], inList[i]
```

Handwritten red annotations: A red 'X' over the variable 'n' in line 2, and a red arrow pointing from the 'n' in line 2 to the range 'i+1, n' in line 6. To the right, a list of indices is written: $n-1$, $n-2$, $n-3$, ..., 1 .

8	5	1	3	4
1	5	8	3	4
1	3	8	5	4
1	3	4	5	8

Handwritten blue annotations: A blue arrow points from the first column (values 1, 1, 1, 1) to the right. Another blue arrow points from the last column (values 4, 4, 4, 8) to the right.

Repeating the length of the list times:

1. Find the i-th minimum value

2. Swap it with the value at index i

What is my Big O?

$O(n)$ vs $O(n^2)$

SelectionSort Big O Breakdown



```
1 def selectionSort(inList):  
2     n = len(inList)  
3     nx  
4     for i in range(n):  
5         minindex = i  
6         for j in range(i+1, n):  
7             if inList[j] < inList[minindex]:  
8                 minindex = j  
9  
10        inList[i], inList[minindex] = inList[minindex], inList[i]
```

$O(n)$

$O(1)$

$$O(n^2 + n)$$

$O(n^2)$

Repeating the length of the list times:

1. Find the i-th minimum value

$$\underbrace{(n-1)}_{\sim n} + \underbrace{(n-2)}_{\sim n} + \underbrace{(n-3)}_{\sim n} + \dots + \underbrace{4}_{\sim n} = \frac{n(n-1)}{2} \sim O(n^2)$$

2. Swap it with the value at index i

$$\hookrightarrow O(1)$$

InsertionSort

Lets make a sorted list by inserting all items into a 'sorted' list one at a time!

Which of the following lists is sorted in ascending order?

A) [5]

C) [5, 1, 4]

B) [5, 1]

D) [5, 1, 4, 9]

InsertionSort



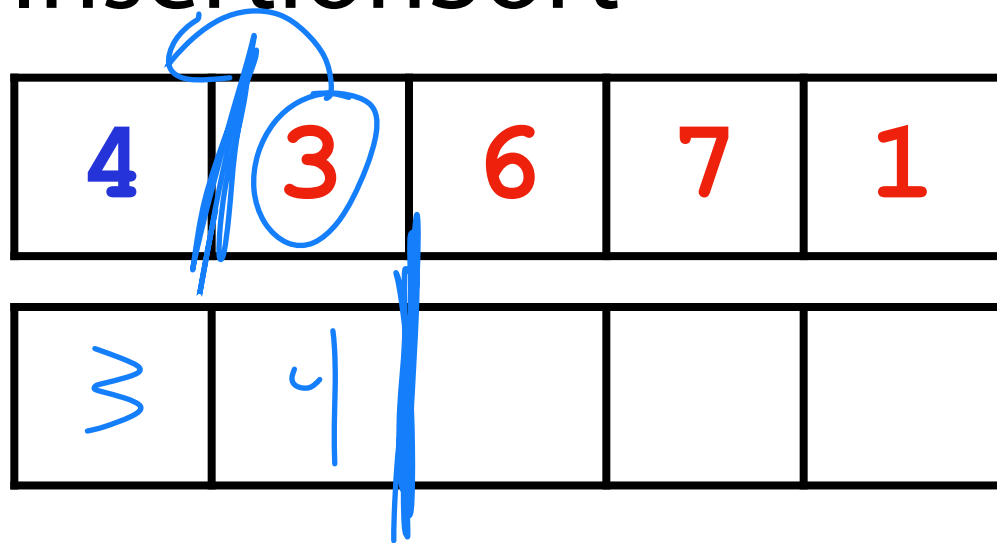
4	3	6	7	1
---	---	---	---	---

1. Divide array into two parts

The 'sorted' array starts as the first item

The 'unsorted' array is everything else

InsertionSort



1. Divide array into two parts

The 'sorted' array starts as the first item

The 'unsorted' array is everything else

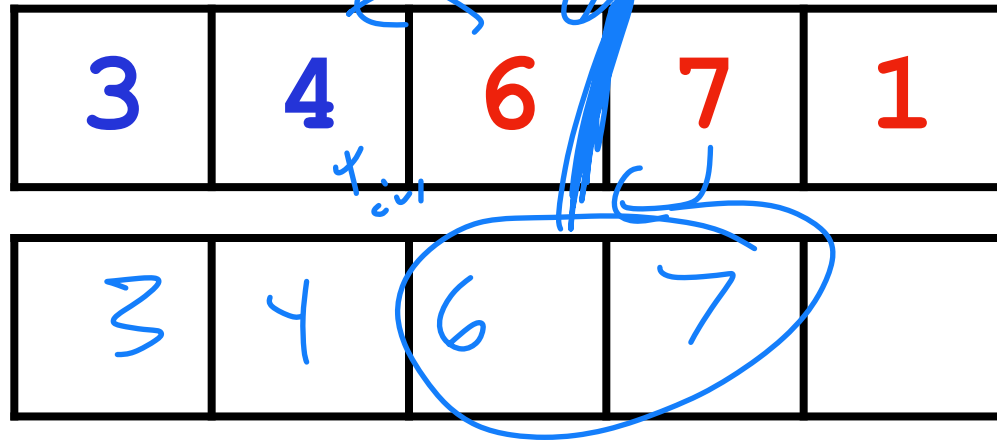
2. Insert each item into the sorted list

Use a sorted array insertion strategy

Insertion strategy for item at index i:

Compare value to item at index $i-1$. If smaller, swap!

InsertionSort i



1. Divide array into two parts

The 'sorted' array starts as the first item

The 'unsorted' array is everything else

2. Insert each item into the sorted list

Use a sorted array insertion strategy

Insertion strategy for item at index i :

Compare value to item at index $i-1$. If smaller, swap!

InsertionSort $i-1$ i

3	4	6	7	1
---	---	---	---	---

1	3	4	6	7
---	---	---	---	---

$x =$
↑

1. Divide array into two parts

The 'sorted' array starts as the first item

The 'unsorted' array is everything else

2. Insert each item into the sorted list

Use a sorted array insertion strategy

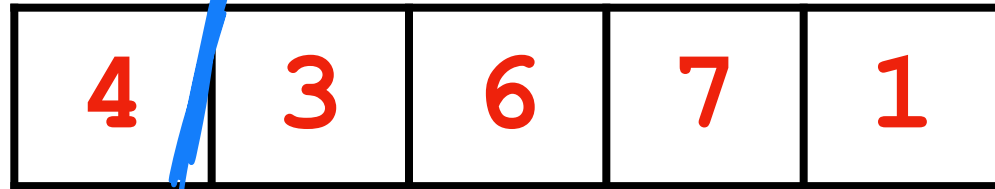
Insertion strategy for item at index i :

Compare value to item at index $i-1$. If smaller, swap!

we are at
↓ correct spot!

Repeat this process until either $i=0$ or $x[i-1] \leq x[i]$

InsertionSort



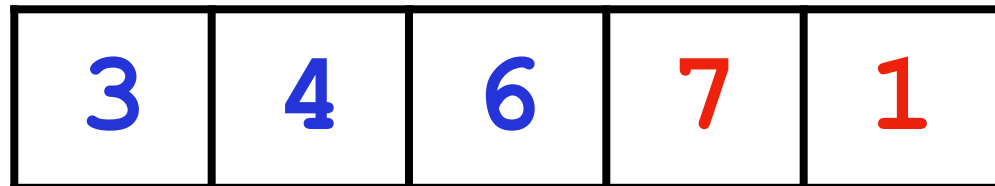
4	3	6	7	1
---	---	---	---	---



3	4	6	7	1
---	---	---	---	---



3	4	6	7	1
---	---	---	---	---



3	4	6	7	1
---	---	---	---	---



3	4	6	7	1
---	---	---	---	---

1. Divide array into two parts

$O(1)$

The 'sorted' array starts as the first item

The 'unsorted' array is everything else

2. Insert each item into the sorted list

Use a sorted array insertion strategy

What is our Big O?

$n \times O(1)$

InsertionSort Insertion Strategy



1	2	4	5	7	3	8
---	---	---	---	---	---	---

1	2	4	5	3	7	8
---	---	---	---	---	---	---

1	2	4	3	5	7	8
---	---	---	---	---	---	---

1	2	3	4	5	7	8
---	---	---	---	---	---	---

Given item at index i being inserted

Compare value to item at index $i-1$,

If smaller, swap and repeat @ $i-1$!

Stop when $i=0$ or index i is larger

Time to do a single insert:

$$X = O(n)$$

How many inserts:

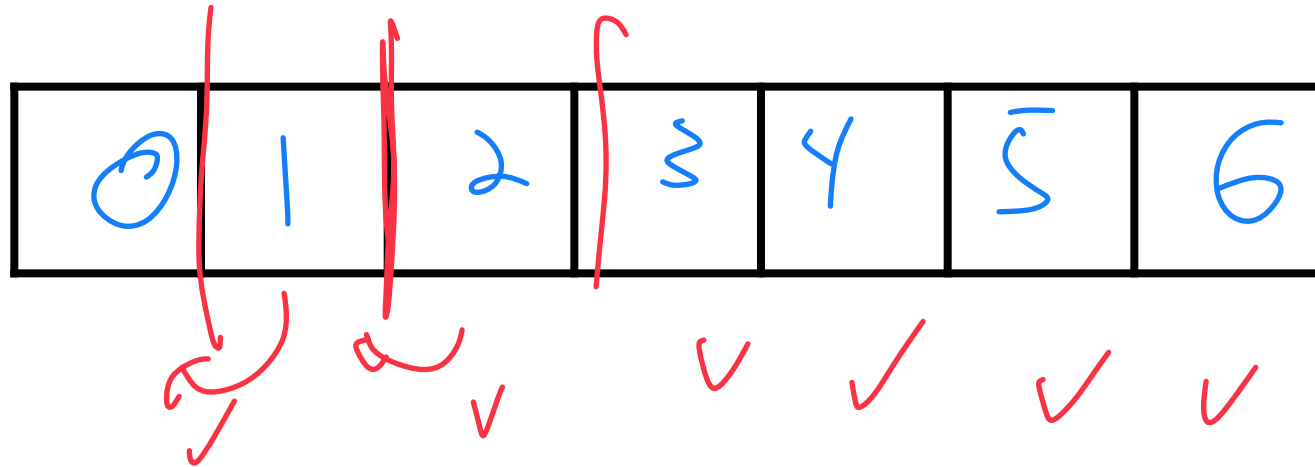
$$Y = O\left(\frac{n \times (n-1)}{2}\right) = O(n^2)$$

4/

$$O(X \times Y)$$

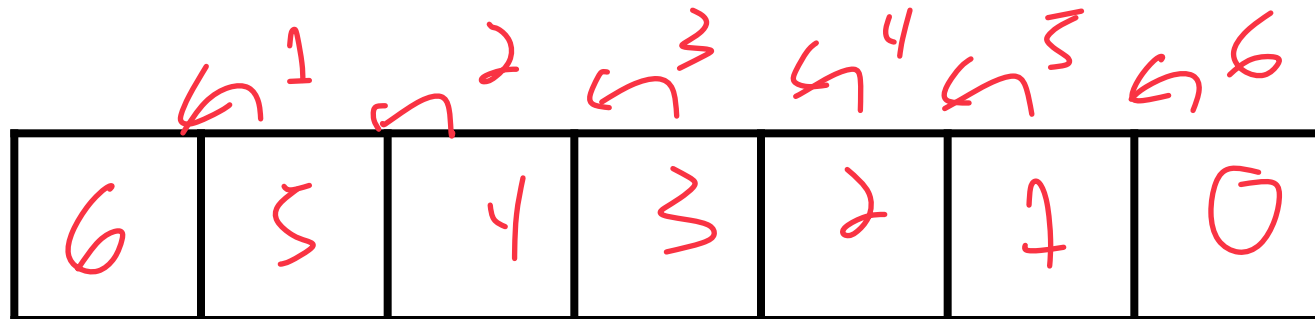
Best and Worst Case insertionSort

Given the numbers 0 — 6, what is the **best** possible insertionSort input?



$$O(n)$$

Given the numbers 0 — 6, what is the **worst** possible insertionSort input?



$$(n-1) + (n-2) + (n-3) + \dots + 1 = O(n^2)$$

Selection vs InsertionSort

What is good or bad about each of these?

Selection Sort $O(n^2)$
Sort in place

Insertion Sort $O(n^2)$
Sort in place

Ways
→
better

If the list is sorted
or almost sorted $O(n)$

Sorting Algorithm Tradeoffs (So far)



	Best Case Time	Worst Case time	Best Case Space	Worst Case Space
SelectionSort	$O(n^2)$	$O(n^2)$	$O(1)$	$O(1)$
InsertionSort ☺	$O(n)$	$O(n^2)$	$O(1)$	$O(1)$
QuickSort				
MergeSort				

QuickSort

6	1	0	3	7	9	2	4
---	---	---	---	---	---	---	---

1. Choose a pivot value

QuickSort

6	1	0	3	7	9	2	4
---	---	---	---	---	---	---	---

< 4

1 0 3 2

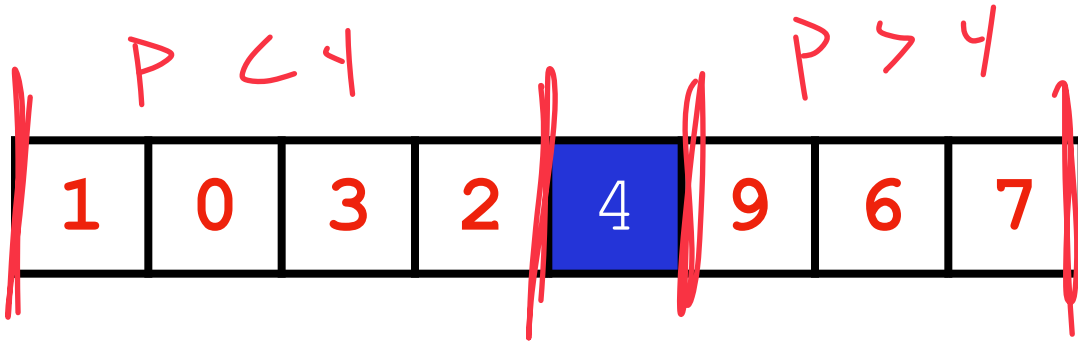
> 4

6 7 9

1. Choose a *pivot* value

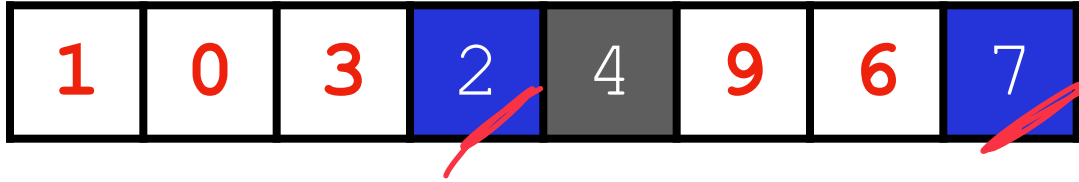
2. Divide the array into two partitions (larger and smaller than pivot)

QuickSort



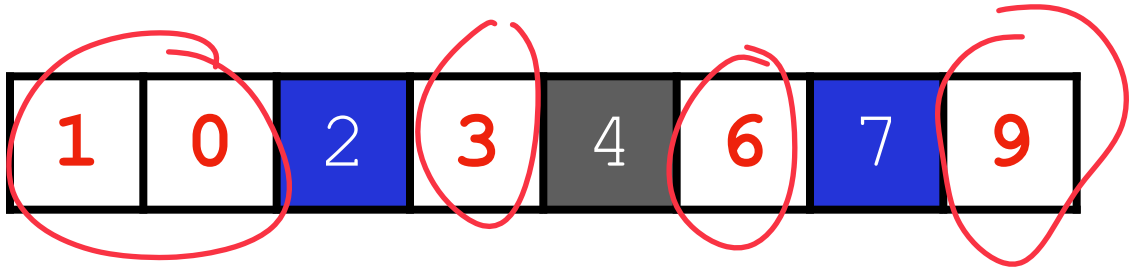
1. Choose a *pivot* value
2. Divide the array into two partitions (larger and smaller than pivot)
3. QuickSort both partitions

QuickSort



1. Choose a *pivot* value
2. Divide the array into two partitions (larger and smaller than pivot)
3. QuickSort both partitions

QuickSort



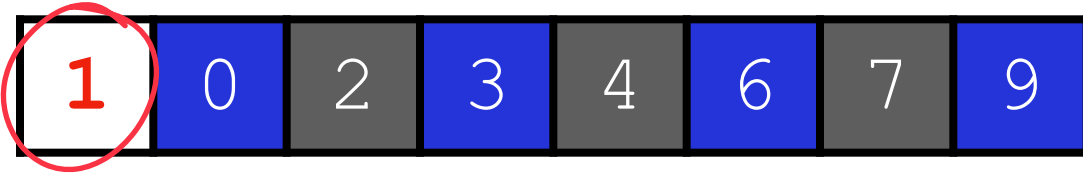
1. Choose a *pivot* value

2. Divide the array into two partitions (larger and smaller than pivot)

3. QuickSort both partitions

Do it again...

QuickSort



1. Choose a *pivot* value

2. Divide the array into two partitions (larger and smaller than pivot)

3. QuickSort both partitions

Do it again...

And again...

QuickSort



6	1	0	3	7	9	2	4
1	0	3	2	4	9	6	7

1	0	3	2
1	0	2	3

9	6	7
6	7	9

1	0
0	1

3
3

6
6

9
9

0	1	2	3	4	6	7	9
---	---	---	---	---	---	---	---

1. Choose a pivot value

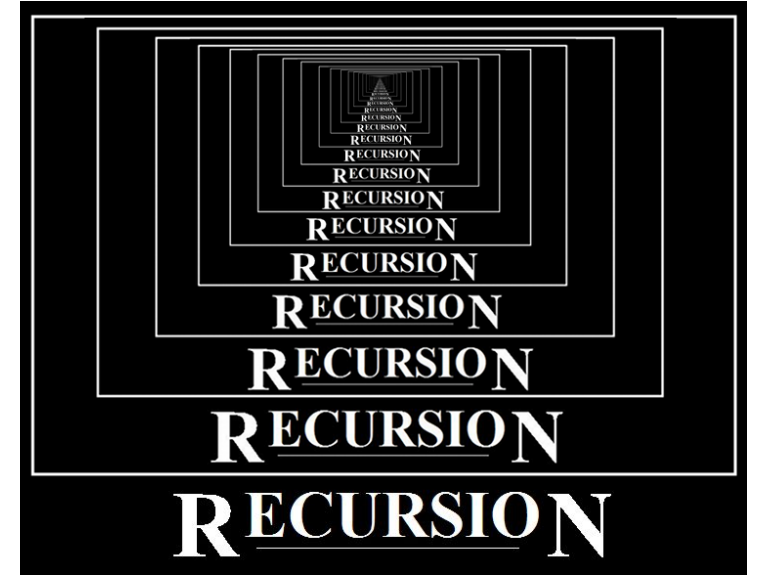
2. Divide the array into two partitions (larger and smaller than pivot)

3. Recursively QuickSort partitions



Programming Toolbox: Recursion

The process by which a function calls itself directly or indirectly is called **recursion**.



When is this useful?

Recursion is a great tool when we can split a problem into two parts:

1) A single 'easy' step or sub-problem

2) 'Everything else' — a slightly smaller sub-problem

Linked List Find

An **iterative** solution is one that loops over all objects:



curr = head

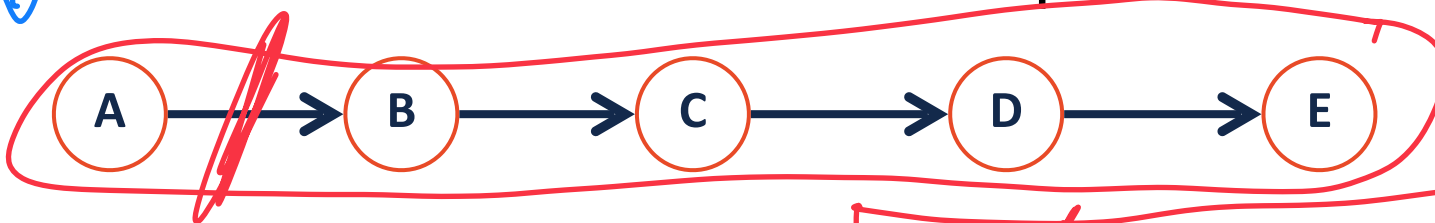
*while curr:
curr = curr.next*



curr



A **recursive** solution breaks the problem into simple / the rest:



Query?

- everything else



If not, search:

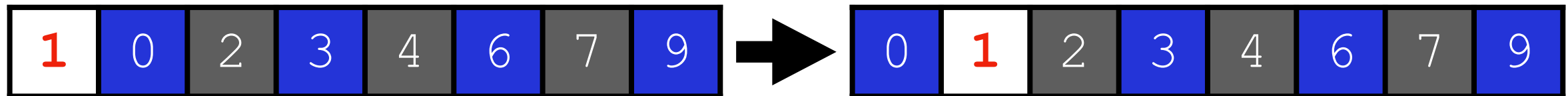
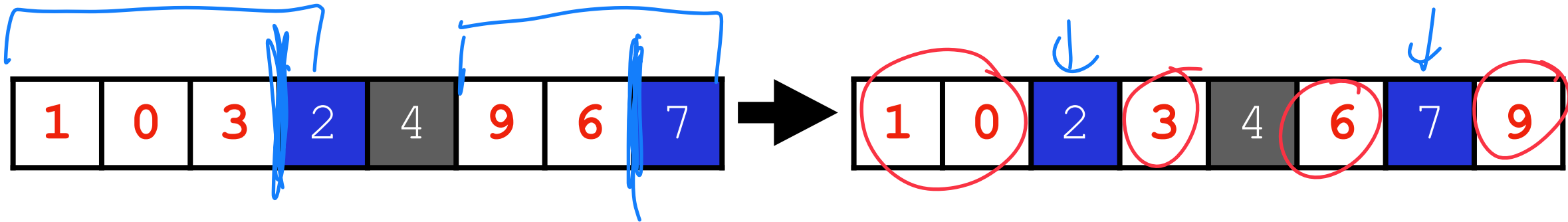
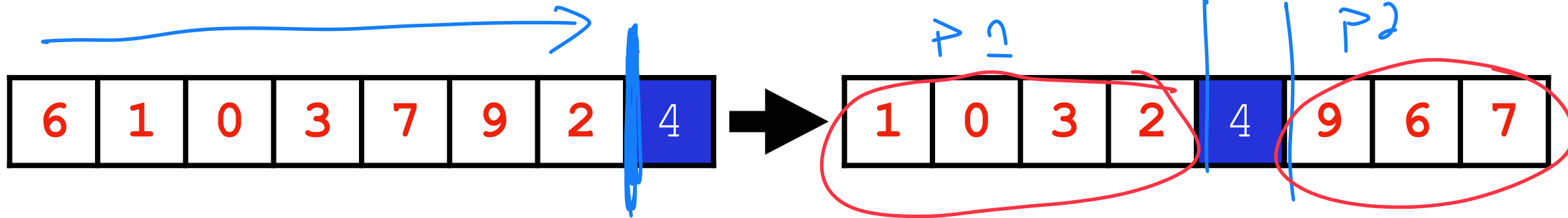


If not, search:



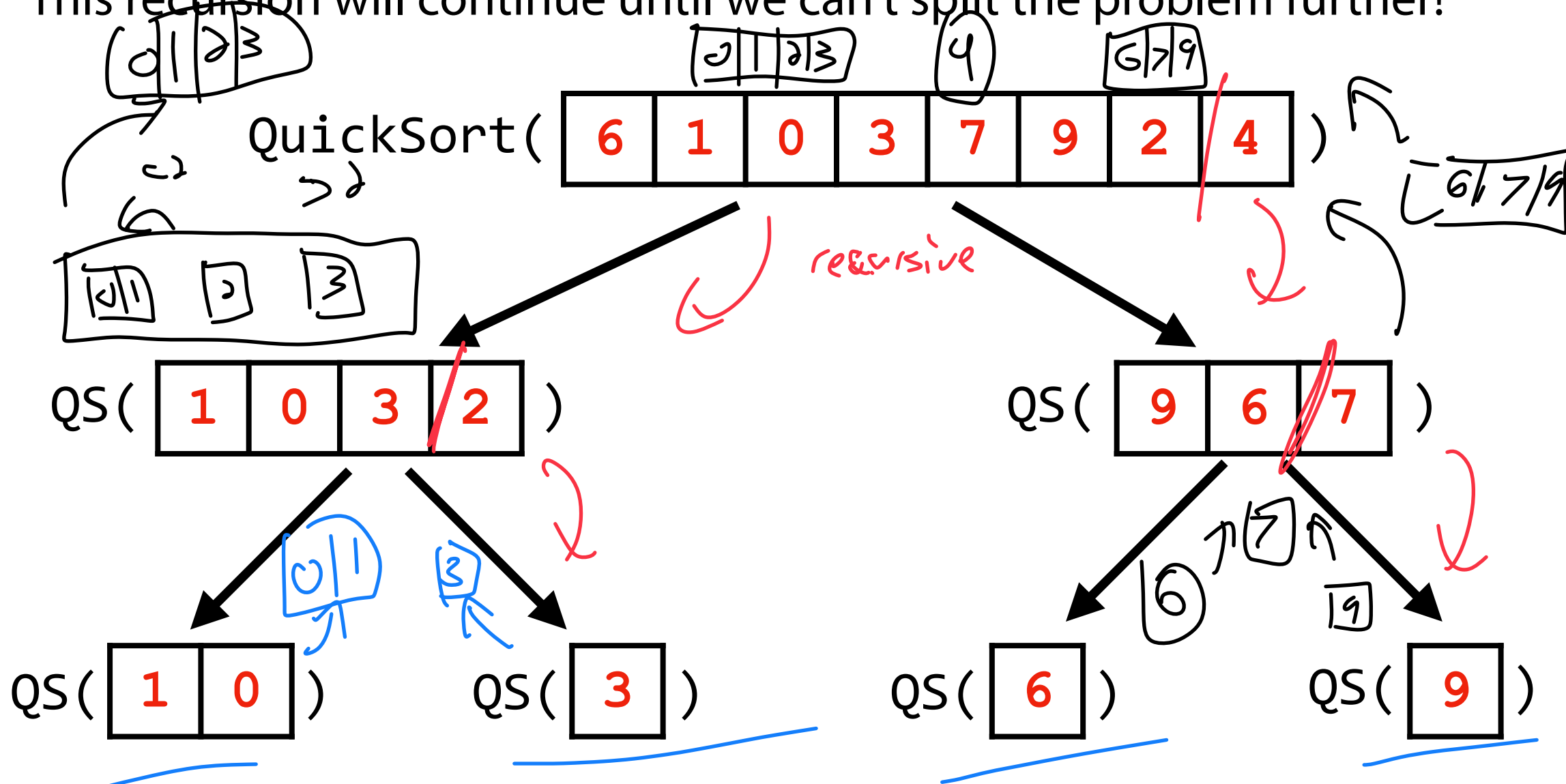
QuickSort Recursion

Quicksort is a **recursive** function! Given one array, it calls quickSort on two!



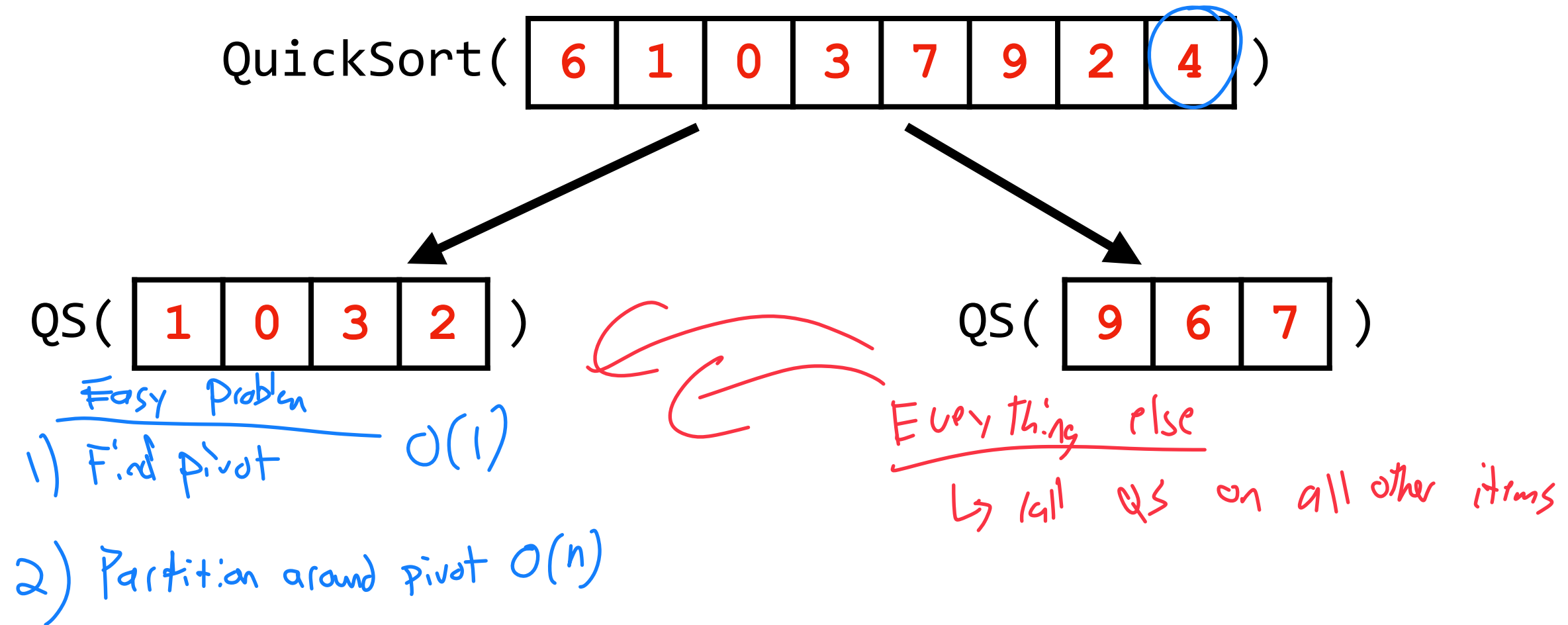
QuickSort Recursion

This recursion will continue until we can't split the problem further!



QuickSort Recursion

What is our 'easy' problem and 'the rest' in the context of quickSort?



Next Class: Recursion more formally

There is a more formal approach for recursion than 'simple' / 'the rest':

Base Case: What is the smallest sub-problem? What is the trivial solution?

Recursive Step: How can I reduce my problem to an easier one?

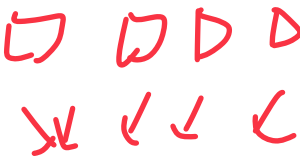
Combining: How can I build my solution from recursive pieces?

Programming Toolbox: Recursion

The more formal approach to 'simple' / 'the rest' is to define the following:

Base Case: What is the smallest sub-problem? What is the trivial solution?

Recursive Step: How can I reduce my problem to an easier one?



Combining: How can I build my solution from recursive pieces?

Recursive Quicksort

~~If you want to practice ahead, try to describe:~~

Base Case:

Trivial soln

When my input is one item
it is sorted!

Recursive Step:

Reduce problem to make a slightly easier one
↳ Find a pivot / partition around pivot

Combining:

Add sorted list of < pivot then pivot then > pivot

